GOSIM

GOSIM Paris 2026

RustNN: A Rust Ecosystem for the Web Neural Network Standard

Markus Tavenrath · GoSim Paris 2026

WebNN: One Graph → Any Hardware

WebNN is the W3C standard for neural network inference

- You describe your computation via MLGraphBuilder.
- The Browser compiles and dispatches it to the best hardware.
  - CPU, GPU, NPU — same graph*
  - Browser delegates to Windows ML · CoreML · LiteRT
- Origin trials started in Chromium and Edge

One graph builder. The platform handles the rest.
But it's browser-only.

executing-an-MLGraph-using-MLTensors.js

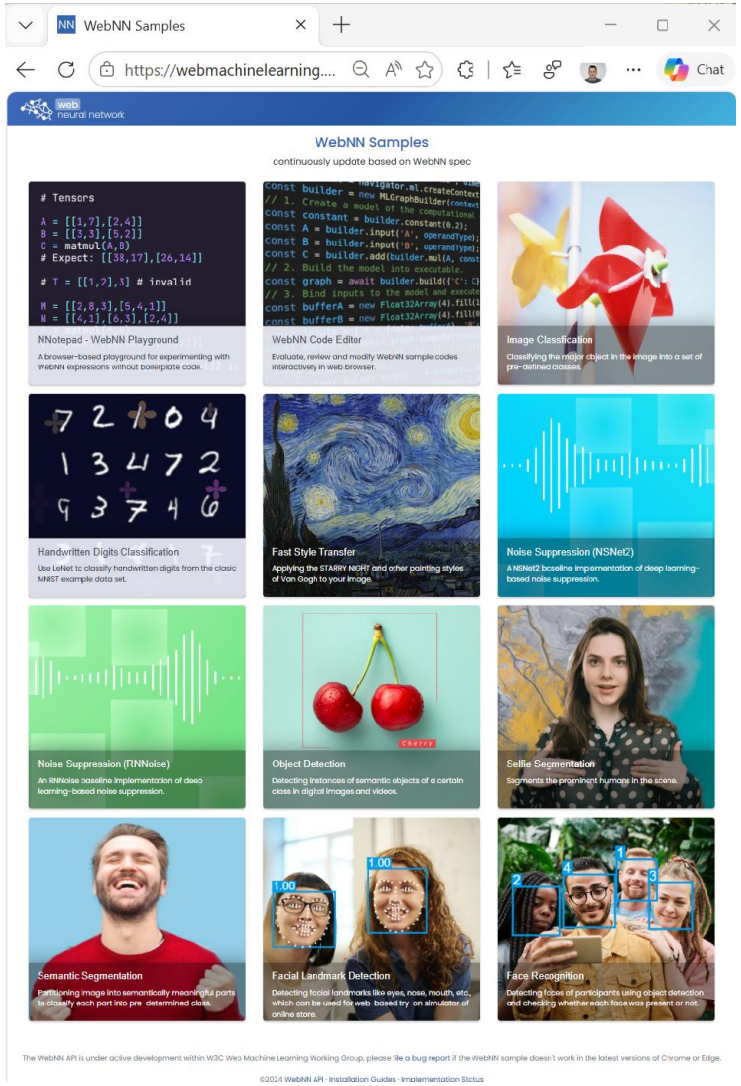
```js
const descriptor = {dataType: 'float32', shape: [2, 2]};
const context = await navigator.ml.createContext();
const builder = new MLGraphBuilder(context);

// 1. Create a computational graph 'C = 0.2 * A + B'.
const constant = builder.constant(descriptor, new Float32Array(4).fill(0.2));
const A = builder.input('A', descriptor);
const B = builder.input('B', descriptor);
const C = builder.add(builder.mul(A, constant), B);

// 2. Compile the graph.
const graph = await builder.build({'C': C});

// 3. Create reusable input and output tensors.
const [inputTensorA, inputTensorB, outputTensorC] =
    await Promise.all([
      context.createTensor({
        dataType: A.dataType, shape: A.shape, writable: true
      }),
      context.createTensor({
        dataType: B.dataType, shape: B.shape, writable: true
      }),
      context.createTensor({
        dataType: C.dataType, shape: C.shape, readable: true
      })
    ]);

// 4. Initialize the inputs.
context.writeTensor(inputTensorA, new Float32Array(4).fill(1.0));
context.writeTensor(inputTensorB, new Float32Array(4).fill(0.8));

// 5. Execute the graph.
const inputs = {
  'A': inputTensorA,
  'B': inputTensorB
};
const outputs = {
  'C': outputTensorC
};
context.dispatch(graph, inputs, outputs);

// 6. Read back the computed result.
const result = await context.readTensor(outputTensorC);
console.log('Output value:', new Float32Array(result));  // [1, 1, 1, 1]
```

Source: https://webnn.io/en/learn/get-started/quickstart

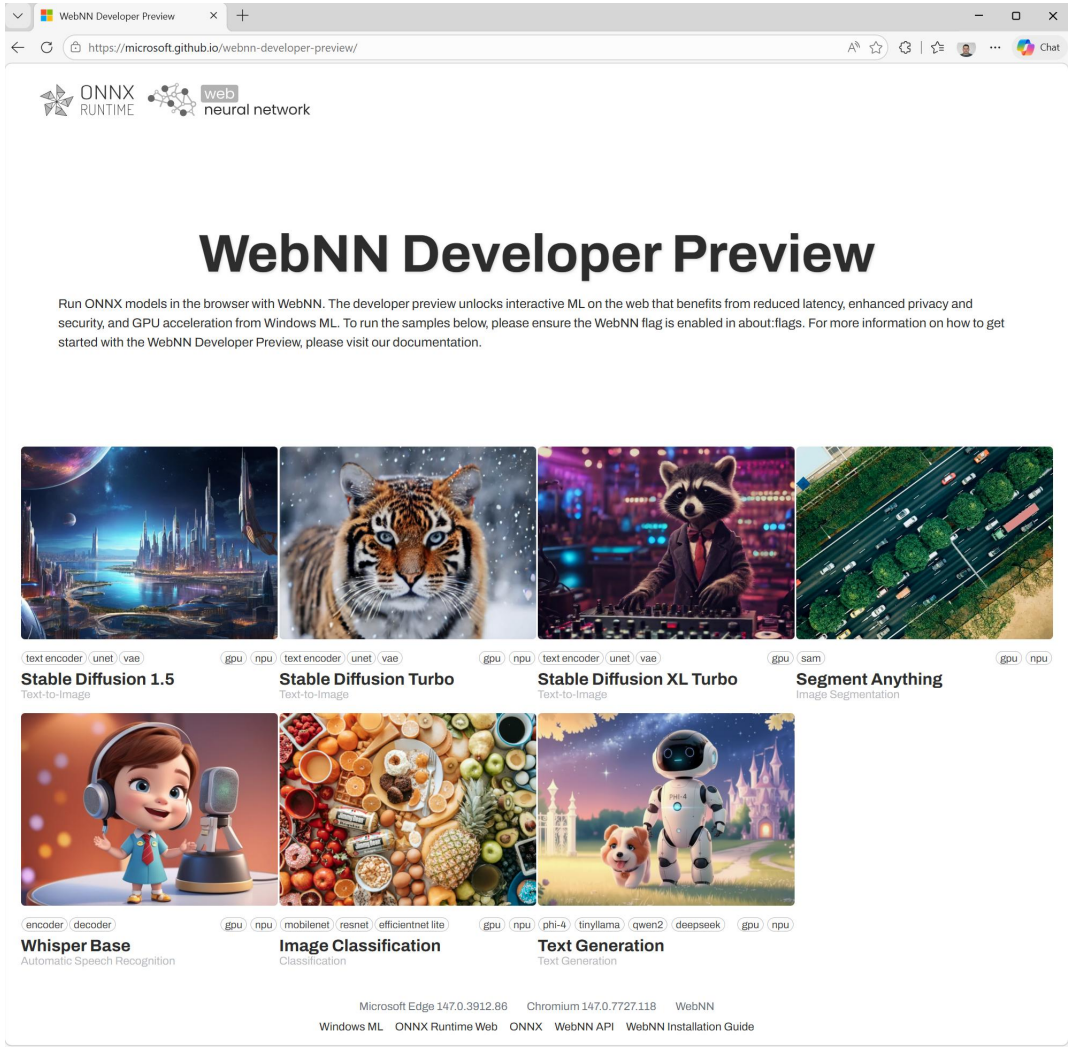
WebNN: One Graph → Any Hardware

Why not just WebGPU or WebAssembly?

- With WebGPU/WASM, getting something functional is straightforward
- But browsers run on every device with a screen — desktop, mobile, low-power, NPU-equipped
- Getting performance portability across that entire landscape is a different problem entirely
  - NPUs are not supported at all
- WebNN delegates that to the platform: vendor-tuned kernels, NPU scheduling — handled by the OS, not you



https://webmachinelearning.github.io/webnn-samples/



https://microsoft.github.io/webnn-developer-preview/

Real workloads which can be targeted by WebNN

Video conferencing	- concurrent ML, audio/video, background removal, noise removal
Object detection	- YOLO12n (real-time)
Vision	- MODNet · Segment Anything · Depth Anything v2
Image generation	- Stable Diffusion Turbo
Speech	- Recognition: Whisper · Generation: Kokoro · Piper
On-device SLMs	- Qwen2.5 · gpt2 · llama-2-7b

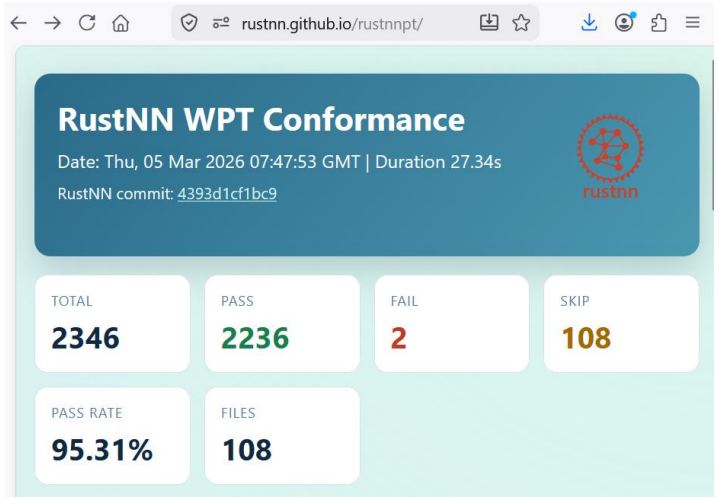
Browser as only platform?

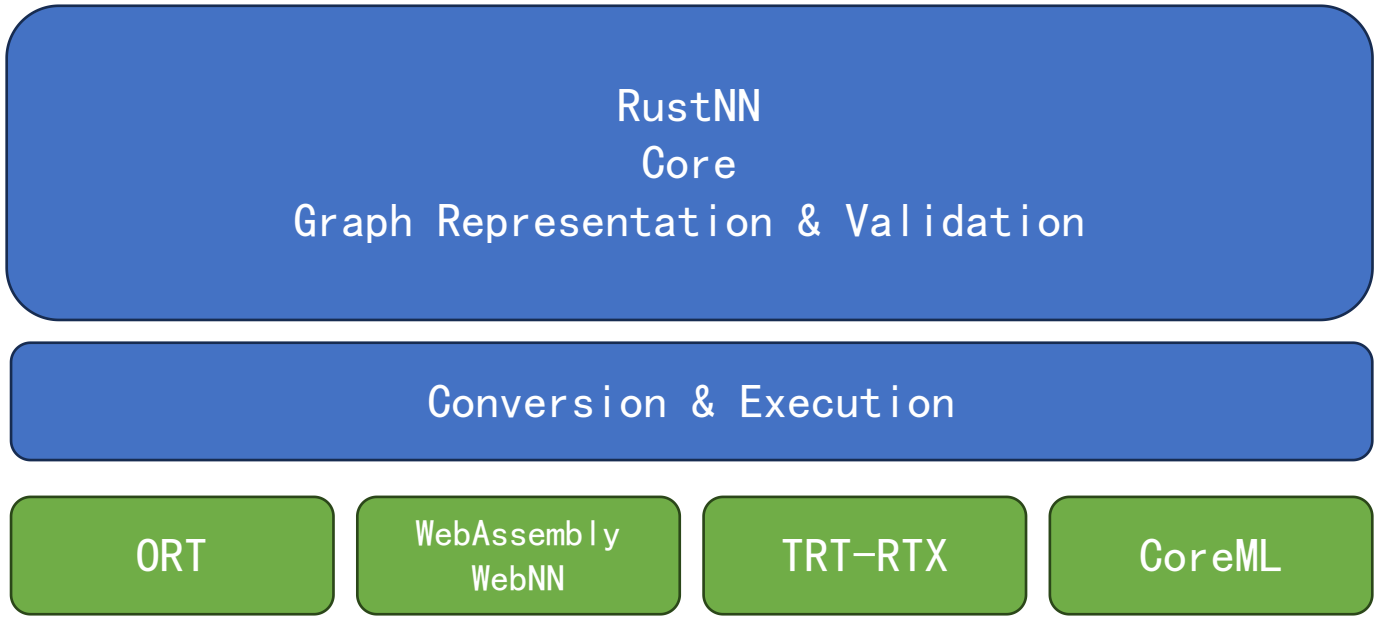
If WebNN works great in the browser, can it work well on the desktop as well?

If it works well, can we build a shared ecosystem?

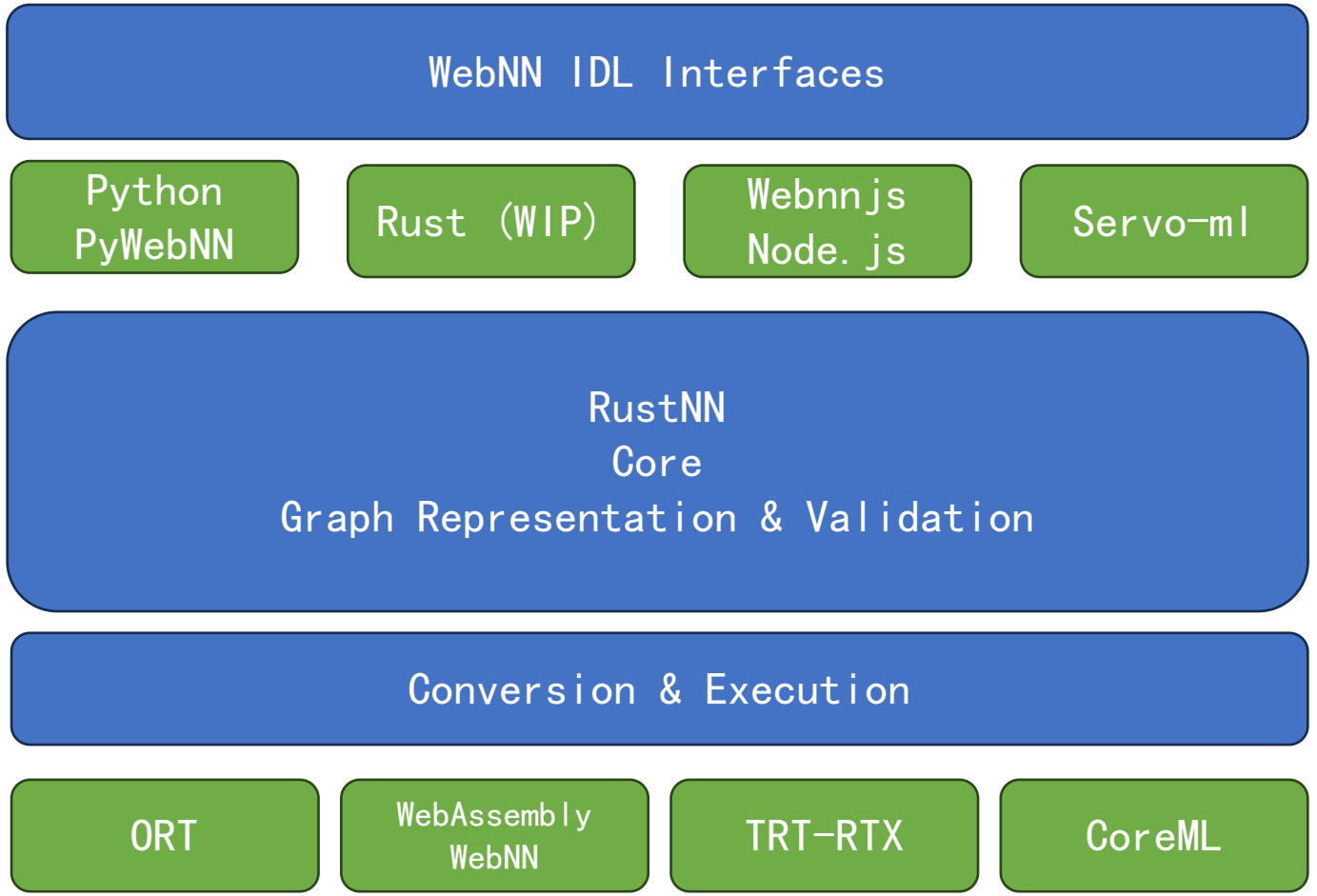
- Chromium has a pretty good WebNN implementation on top of ONNX Runtime & LiteRT
- Unlike WebGPU (Dawn) their implementation is deeply intertwined with Chromium
- What about other browsers or native applications?
  - rustnn - a Python and Rust Implementation of W3C WebNN aimed at Firefox | Tarek Ziadé
  - Work started around December 2025.
- I joined the project in January after realizing its potential
  - A native implementation would allow for a single deployment path in the web and desktop
  - It is designed to run on any other important AI Inference API
- A full ecosystem flow is required to get the developer where he is today
  - Python!

- RustNN core
  - Everything you need to implement the WebNN standard
- Tested through rustnnpt, a rust wrapper to run the WebNN WPT Tests
  - A subset of the tests does not run yet due to the bindings





- WebNN IDL language bindings
  - Implemented on top of RustNN
  - Python, Node.js are fine
  - Rust is work in progress
  - Servo-ml is a servo fork
    - Adds WebNN support
  - C/C++ are under planning

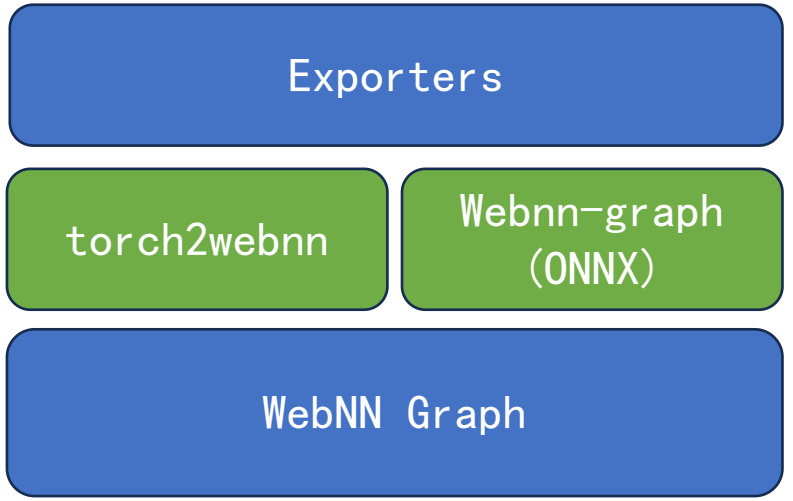


PyWebNN

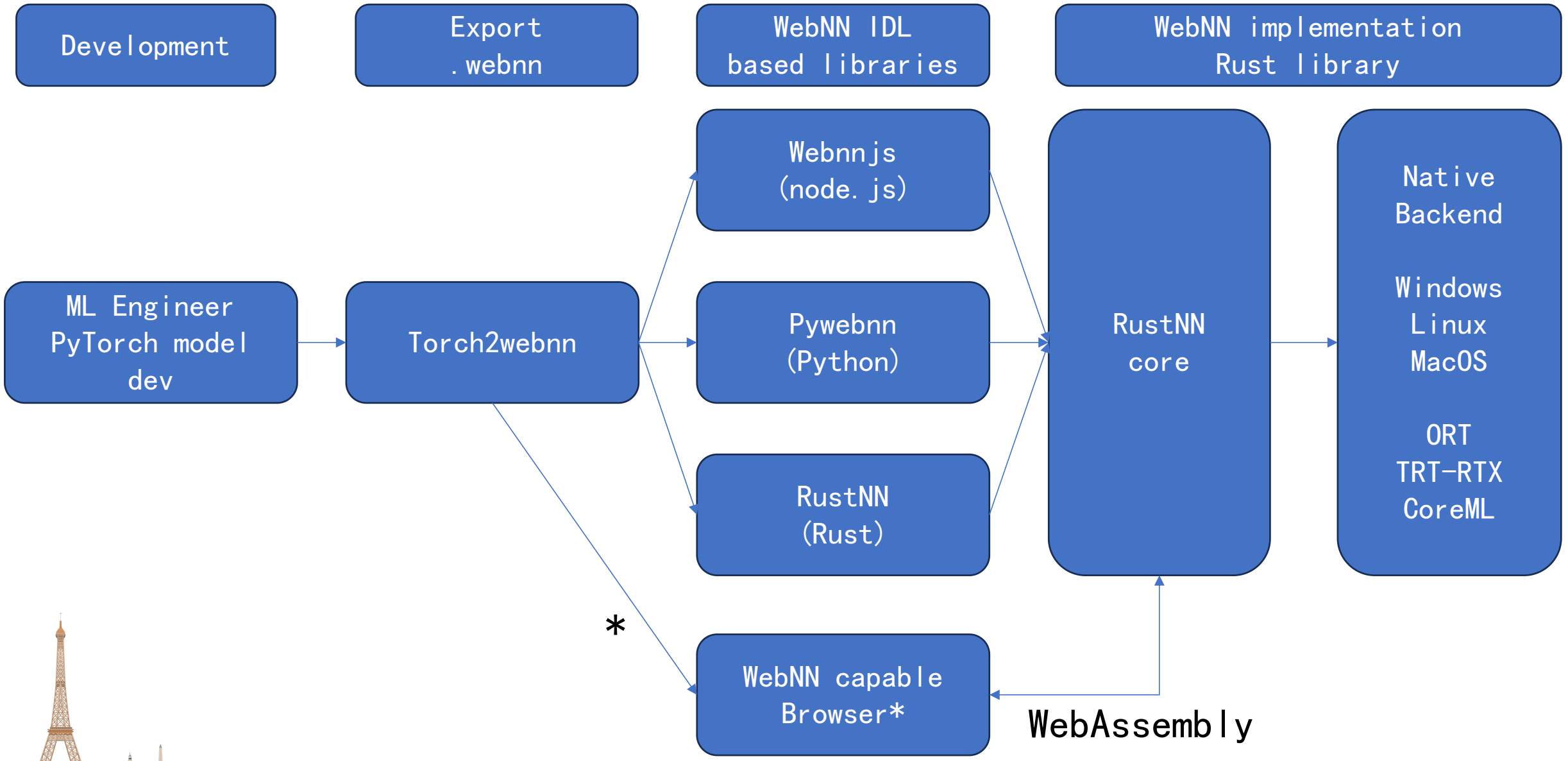
- Develop network in Python
- Use Torch / Numpy to verify math
- Once satisfied call graph.save(“graph.webnn”)

```python
import numpy as np
import webnn

ml = webnn.ML()
context = ml.create_context(device_type="cpu")
builder = context.create_graph_builder()

x = builder.input("x", [2, 3], "float32")
y = builder.input("y", [2, 3], "float32")
out = builder.relu(builder.add(x, y))
graph = builder.build({"output": out})

result = context.compute(
    graph,
    {
        "x": np.array([[1, 2, 3], [4, 5, 6]], dtype=np.float32),
        "y": np.array([[0.1, 0.2, 0.3], [0.4, 0.5, 0.6]], dtype=np.float32),
    },
)
print(result["output"])
```

- How do we get content into the ecosystem?
  - RustNN ecosystem should be fully self-contained.
- Required projects
  - webnn-graph
    - A graph DSL & rust-based library to load and save the DSL.
    - Designed to be close to the WebNN graph
    - Can use .safetensors for weights
  - onnx2webnn (currently part of webnn-graph)
    - Early phase, only basic models supported
    - Goal: Support broad ONNX ecosystem
  - torch2webnn
    - Early phase, used to export demo model
    - Converts Aten IR to webnn-graph
    - Goal: As lossless as possible PyTorch -> WebNN export



Workflow – Export once, run anywhere



What's Next

- Conformance
  - Ensure we can run 100% of the conformance tests for better test coverage
- Browser integration
  - Update Firefox integration prototype
  - This gives us more content for testing and benchmarks
- Generic GPU Backend?
  - WebGPU has experimental support for subgroup matrices (TensorCores)
    - Supported by Vulkan, Metal and D3D12 though LinAlg Matrix Preview
  - Use ORT-WebGPU or Burn? Burn is more flexible (CPU, Vulkan, CUDA, Metal, …)
- Content!
  - Export more models through torch2webnn
    - Harden the exporter and backends

Get Involved

GitHub: http://github.com/rustnn
Try it: pip install pywebnn — experiment with WebNN in the python ecosystem
Test it: run rustnnpt conformance suite
Contribute: file issues, PRs and most important
Use it: Develop networks and demos to show what is possible with WebNN

GOSIM

GOSIM Paris 2026

WebNN: one graph → any hardware — but locked in the browser.

RustNN brings WebNN to the edge: standalone · embeddable · spec-compliant · no browser required.

Train in PyTorch, deploy anywhere — CPU, GPU, NPU, any language, any device.

github.com/rustnn

Questions?